# NED University of Engineering & Applied Sciences

Department of Computer & Information Systems Engineering MS

AI Evening Program | Spring 2026

**CS-5101**

**Advanced Artificial Intelligence**

## ASSIGNMENT 1 & 2

**Build Karachi's Smartest Transport Planner**

### ⬖⬖ Assignment Details

Format: Design Project (Report + Working Code)

Teams: Groups of 2

Language: Python

Covers: Lectures 1 through 4

Instructor: Muhammad Mudassir Majeed

# ⬥⬥ The Challenge

### The Story

Karachi is one of the largest cities in the world — and one of the hardest to get around. Millions of people struggle with unpredictable routes, missing connections, and a transport network that feels like it was designed by rolling dice on a map.

Imagine you've been brought in to fix this. Your mission: design an AI-powered transport planning system that can find the best routes for passengers AND figure out where bus stops and routes should actually go.

You'll start by understanding the problem deeply, then build progressively smarter solutions — from brute-force search all the way to intelligent optimization.

## What You'll Build

Your system has two brains:

### Brain 1 — The Route Finder

A passenger asks: "How do I get from Saddar to Korangi?"

Your system finds the best route. You'll make this progressively smarter across Layers 1 and 2.

### Brain 2 — The Network Designer

The city asks: "Where should we put bus stops and routes to serve the most people?"

Your system optimizes the entire network layout. This is Layer 3 — a completely different kind of AI problem.



**CS-5101 | Build** Karachi's Smartest Transport Planner

## How the Layers Build on Each Other

Each layer of this assignment builds on the previous one. Don't throw away code — extend it. Think of it as building a product, not answering separate questions.

| | | | |
|---|---|---|---|
| Part A | Analyse & formulate the problem | Agents, PEAS, | 1 & 2 |

| | | | |
|---|---|---|---|
| | | environments, state representation | |
| Layer 1 | Build the graph + basic route finding | BFS, DFS, UCS | 2 |
| Layer 2 | Add heuristics for smart routing | Greedy, A*, IDA*/RBFS/SMA* | 3 |
| Layer 3 | Optimize the whole network | Hill Climbing, Simulated Annealing | 4 |



# ⬜⬜ PART A: Understand the Problem First

Before writing a single line of code, understand what you're solving. This is where most real AI projects succeed or fail. Part A is your analysis — pure thinking, clearly written.

## A1. Meet Your Agents

You're actually designing two different AI agents. They have different goals, different inputs, and need different approaches. Think about them separately.

### Agent 1 — The Passenger Route Planner

- What does it see? What does it know about the world?
- What actions can it take?
- How do we know if it did a good job?

### Agent 2 — The Network Designer

Very different from Agent 1! This one isn't finding a path — it's designing a whole system. What does "success" look like here?

What to deliver:

• **PEAS table** for each agent. Make it specific to Karachi — generic answers like "sensors: cameras" won't cut it. Think about what data actually exists and matters in Karachi's context. • **Environment classification** for each agent (deterministic/stochastic, episodic/sequential, fully/partially observable, single/multi-agent, static/dynamic). The two agents might be classified differently — explain why.



## A2. Design Your Karachi

This is a creative step — you're building the map your entire system will run on.

### Design a Graph of Karachi's Transport Network

• At least 20 real Karachi locations as nodes — pick places everyone knows (Saddar, Gulshan, Johar, DHA, Malir, Korangi, North Nazimabad, Orangi, Clifton, SITE, etc.) • Edges represent direct road/route connections — not everything connects to everything. • Edge weights = estimated travel time in minutes. Be realistic.

• Include at least 2–3 bottleneck areas where connectivity is poor (e.g., crossing from Orangi to DHA). These make the search problems interesting.

• Minimum 35 edges total.

• Document your choices — a short paragraph on why you picked these nodes and how you estimated the weights.

Then formulate the route-finding problem:

- **State representation:** Start with atomic (just location), then argue why factored is better. What other variables matter for a real passenger? (time of day? transfers? budget?) Identify at least 5 factored state variables with their types and possible values.
- **Formal search problem:** Write out the complete definition — state space, initial state, actions, transition model, goal test, path cost. Be precise enough that a programmer could implement directly from your spec.
- **Complexity argument:** Estimate the state space size for both atomic and factored representations. Use this to argue why efficient search algorithms matter.

## A3. Your Game Plan

Before coding, write a short strategic overview (roughly one page):

- For route-finding: which search algorithms are candidates? What are the trade-offs? What heuristic could work for Karachi's geography?
- For network optimization: why is this a fundamentally different kind of problem? Why can't A* solve it? What makes local search the right tool?

This section is about connecting theory to your specific problem. Show us you understand why different AI techniques exist.



# ⬛⬛ PART B: Build It, Layer by Layer

Each layer builds on the previous one. Don't throw away code — extend it. By the end, you'll have a working system that demonstrates concepts from all four lectures.

## ⬛⬛ Layer 1: The Basics — Uninformed Search

**Goal:** Build the graph. Find routes. No cleverness yet — just systematic search.

### Build

- Implement your Karachi graph in Python (adjacency list or similar structure). It should support weighted edges and be easily extensible.
- **Implement BFS, DFS (with cycle detection), and UCS** — all sharing a common interface. Same inputs (start, goal), same output format (path, cost, nodes expanded, max frontier size).

## Test

Pick 5 origin-destination pairs that are deliberately interesting:

1. A short, easy direct route
2. A long cross-city route
3. A route through a bottleneck area
4. A route where BFS and UCS find different paths (because edge weights vary) 5. A route where DFS performs particularly badly

Run all three algorithms on all five pairs.

## Report

- **Results table;** path found, path cost, nodes expanded, max frontier size for each algorithm on each pair.
- **Analysis (200–300 words):** When did the algorithms disagree? Why? Where did DFS waste effort? Do the results match the theoretical properties from lecture?



## ⬥⬥ Layer 2: Get Smart — Heuristic Search

**Goal:** Make route-finding dramatically faster and smarter using heuristics.

## Design Two Heuristics

### $h_1$ — Geographic Heuristic

- Based on straight-line distance between locations.
- You'll need rough latitude/longitude coordinates for your nodes — estimate from Google Maps.
- Convert distance to estimated travel time using an assumed average speed.

### $h_2$ — Your Invention

- A more informed heuristic that uses Karachi-specific knowledge.
- Maybe it accounts for known congestion zones, river/nullah crossings, or the fact that north-south travel is slower than east-west.
- Be creative — this is where you show you understand heuristic design.

**For both heuristics:** argue why they're admissible. Check consistency on at least 5 edges.

## Build

- **Greedy Best-First Search** and A* — both should work with either heuristic (pass heuristic as a parameter).
- **One memory-bounded variant** — pick IDA*, RBFS, or SMA*. Justify your choice in the report based on the properties of your graph.

## Test

Run everything (including UCS from Layer 1) on the same 5 origin-destination pairs.



## Report

- **Comparison table + at least one chart** (e.g., bar chart of nodes expanded across algorithms). • How much did A* improve over UCS in nodes expanded?
- Does $h_2$ dominate $h_1$ empirically? (Does it always expand fewer nodes?)
- Did Greedy ever find a non-optimal path? Show a specific example.
- How did your memory-bounded variant compare to standard A*?

## ⬥⬥ Layer 3: Design the Network — Local Search & Optimization

**Goal:** Shift from "find a route on an existing network" to "design the best possible network." This is a completely different problem — and it needs completely different AI.

## The Optimization Problem

- KMC wants to select K bus stops from your N candidate locations and design routes connecting them.
- The goal: maximize coverage and minimize average travel time across the city.
- The path to get to this solution doesn't matter — only the final network configuration does.
- This is why we need local search, not A*.

## Formulate

- **State:** A complete candidate network — which stops are active, which routes connect them.
- **Objective function:** Design a meaningful function that balances coverage (are most areas reachable?), connectivity (can passengers get between major hubs?), and efficiency (how long do trips take?). You decide the weighting — justify it.
- **Neighbourhood function:** What counts as a small change to a network? (Swap a stop? Add/remove a connection?) Discuss the trade-off between sparse and dense neighbourhoods.

## Build Three Algorithms

6. **Hill Climbing** — steepest ascent, with and without sideways moves
7. **Random-Restart Hill Climbing** — using the above as a subroutine
8. **Simulated Annealing** — with a configurable cooling schedule

## Experiment

- **Hill Climbing:** 50 random starts. Report success rate, average objective value achieved, and average steps.
- **Simulated Annealing:** Try at least 3 cooling schedules (e.g., r = 0.90, 0.95, 0.99), 50 runs each.

## Report



- Which algorithm found the best network configuration? Present the best solution —

which stops, which connections, what objective value.

- A visualization of your best solution (a diagram showing chosen stops and routes on a rough Karachi outline — even a simple matplotlib plot is fine).
- Chart: SA performance vs. cooling rate.
- Hill climbing: success rate with vs. without sideways moves.
- **Final recommendation to KMC:** which algorithm would you actually deploy and why?

## ⬥⬥ What You Turn In

| | |
|---|---|
| Design Report | A single PDF covering Part A and Part B. Clear, well-organized, with tables, charts, and diagrams. Think of it as something you'd be proud to show in a job interview. |
| Python Code | Clean, commented, runnable. Include a README explaining how to run each layer. All three layers should build on the same codebase. |
| Graph Data | Your Karachi graph (as a Python file, JSON, or CSV) with documentation of nodes, edges, and weights. |

## ⬥⬥ Grading Breakdown

| | | |
|---|---|---|
| Part A: Analysis & Formulation | ~25% | Sharp thinking. Precise PEAS. Thoughtful state representation. Clear problem formulation. |
| Layer 1: Uninformed Search | ~15% | Correct implementations. Meaningful test cases. Honest analysis of results. |
| Layer 2: Heuristic Search | ~25% | Creative heuristic design. Admissibility arguments. Thorough comparison with charts. |
| Layer 3: Network Optimization | ~25% | Good objective function. Proper SA experimentation. Insightful analysis. |
| Report Quality | ~10% | Clear writing. Good visualizations. Cohesive story from problem to solution. |

### A Tip from Your Instructor

• The best reports tell a story. Start with the problem, show your thought process. Explain what you tried, what worked and what didn't, and end with clear conclusions. • We value honest analysis over perfect results. If hill climbing got stuck 90% of the time, say so — and explain why. That shows understanding.
• Have fun with it! This is your city. You know its roads, its traffic, its chaos. Use that info!

**Good luck and enjoy building something for Karachi!**